

Status

Name: Timothious Gill

Designation : AI/ML Intern

GitLink: [Week 7: Wed](#)

Date: 17/09/2026

Summary

Today I continued working on **Task 29**, which focuses on building a FastAPI service for interacting with both local and hosted Large Language Models (LLMs). The main focus was on improving the service structure, separating configuration from application logic, using a shared model-calling function, integrating OpenAI-compatible clients, and making the service easier to configure and evaluate.

A major part of today's work involved understanding the architecture of the FastAPI service. Both `/chat/local` and `/chat/hosted` endpoints follow the same request and response contract while using the shared `ask_model()` function for model inference. This helped reinforce the concept of separating the API layer from the underlying model provider and inference logic.

I also reviewed how the application selects between local and hosted inference. The local configuration uses **Ollama** as the model runtime, while the hosted configuration uses an OpenAI-compatible API provider. The provider URL, API key, and model name are controlled through environment variables so that the application code does not need to be changed when switching configurations.

Another important part of today's work was understanding the difference between an **API client, API endpoint, provider, model, and local model runtime**. I also reviewed how Ollama allows locally running LLMs to expose an OpenAI-compatible API, which makes it possible to use the same Python client and shared inference logic for both local and hosted models.

I additionally reviewed model quantization concepts in relation to the local **Llama 3.2 3B** model. I understood that the application itself does not perform quantization; instead, the local model/runtime provides the selected model variant. The local Ollama model was reviewed in relation to the **Q4_K_M** quantization format.

Finally, I worked on the project documentation and evaluator setup. I prepared the structure for a `README.md` file containing the correct Uvicorn command, project structure, installation instructions, environment configuration, API endpoints, testing instructions, and local model requirements. I also continued reviewing the unit tests and the PEFT/LoRA documentation.

Task 1 – FastAPI Service Refactoring

- Reviewed and refined the FastAPI application structure.
- Used `app/main.py` as the main application module.
- Kept separate endpoints for local and hosted inference:
 - `/chat/local`
 - `/chat/hosted`

- Used a shared `ask_model()` function for the actual model-calling logic.
- Learned how shared logic reduces duplication between endpoints.
- Kept the API request and response structure consistent across both endpoints.
- Used Pydantic models for request and response validation.
- Used `ChatRequest` to validate incoming messages.
- Used `ChatResponse` to define the response structure.
- Added a `/health` endpoint for checking whether the FastAPI application is responding.

Task 2 – Local and Hosted Model Architecture

- Reviewed the architecture of the local and hosted inference flow.
- Understood that the FastAPI endpoint acts as the application/API layer.
- Understood that the model provider is responsible for serving the actual model.
- Reviewed the difference between:
 - FastAPI application
 - API endpoint
 - API client
 - Model provider
 - Model
 - Local model runtime
- Understood that the client is responsible for communicating with the provider, while the model name identifies which model should perform the inference.
- Learned that the same application contract can be maintained even when the underlying provider changes.

The basic flow is:

FastAPI → Endpoint → `get_client()` → `ask_model()` → Model Provider → Model → Response

Task 3 – OpenAI-Compatible Client Integration

- Used the OpenAI Python client for model communication.
- Reviewed how `base_url` determines where the API request is sent.
- Reviewed how `api_key` is used for authentication where required.
- Reviewed how the `model` parameter identifies the model to be used.
- Learned that an OpenAI-compatible API allows different providers to expose a similar request format.
- Used the same `ask_model()` function for both local and hosted model requests.
- Learned how this approach reduces provider-specific application code.

The shared model call uses the standard chat-completion structure:

```
client.chat.completions.create(...)
```

Task 4 – Environment-Based Configuration

- Reviewed the use of `.env` for application configuration.
- Configured separate variables for local and hosted inference.
- Local configuration includes:
 - `LOCAL_BASE_URL`
 - `LOCAL_API_KEY`
 - `LOCAL_MODEL`
- Hosted configuration includes:
 - `HOSTED_BASE_URL`
 - `HOSTED_API_KEY`
 - `HOSTED_MODEL`
- Used `python-dotenv` to load environment variables.
- Reviewed why API keys should not be hardcoded inside source code.
- Learned how environment-based configuration makes it easier to change providers or models without modifying the FastAPI implementation.
- Reviewed the use of `override=True` when loading `.env` values.

Task 5 – Ollama and Local LLM Inference

- Reviewed Ollama as the local LLM runtime used by the application.
- Understood that **Ollama is not the LLM itself**.
- Identified the different components:
 - **Ollama:** local model runtime/API server
 - **Llama 3.2 3B:** language model
 - **localhost:11434/v1:** local API endpoint
 - **OpenAI Python client:** client used to communicate with the API
 - **FastAPI:** application layer exposing the project endpoints
- Reviewed the local configuration:

`http://localhost:11434/v1`

- Understood that `localhost` refers to the local machine.
- Reviewed port `11434` as the Ollama service port.
- Reviewed `/v1` as the OpenAI-compatible API path used by the application.
- Understood why using Ollama allows the application to communicate with a local model without directly managing model loading and generation inside the FastAPI code.

Task 6 – Model Quantization

- Reviewed the concept of model quantization for local LLM inference.
- Inspected the local Llama 3.2 3B model information through Ollama.
- Reviewed the **Q4_K_M** quantization format.
- Learned that quantization reduces the numerical precision used to represent model parameters.

- Understood that lower-precision model variants can reduce memory requirements and make local inference more practical.
- Learned that the application code itself does not perform the quantization.
- Understood that Ollama provides/manages the selected model variant.
- Learned that Q4_K_M should not simply be described as every parameter being a basic INT4 value because it is a block-based quantization format with different precision characteristics.

Task 7 – Error Handling

- Reviewed error handling for local and hosted model requests.
- Used `try/except` around downstream model calls.
- Returned HTTP status code **502 Bad Gateway** when the FastAPI service encounters an upstream model/provider failure.
- Learned why 502 is appropriate for failures involving a downstream model service or provider.
- Used `from exc` to preserve the original exception as the cause.
- Reviewed how configuration errors, provider errors, connection failures, and model errors can be surfaced through the API.

Task 8 – Testing

- Reviewed the testing structure under the `tests/` directory.
- Reviewed `tests/test_ask_model.py`.
- Focused testing around the shared `ask_model()` function.
- Continued preparing the remaining unit-test work for the refactored service.
- Learned the importance of testing shared model-calling logic separately from the API endpoint layer.
- Prepared the project to be tested using:

`pytest`

- Reviewed how separating model logic from endpoint logic makes the service easier to test.

Task 9 – PEFT and LoRA Documentation

- Continued working on the PEFT documentation for the task.
- Reviewed the purpose of **Parameter-Efficient Fine-Tuning (PEFT)**.
- Reviewed **LoRA (Low-Rank Adaptation)** as a parameter-efficient fine-tuning approach.
- Continued preparing `peft_lora_explanation.md`.
- Focused on documenting the concepts clearly so they can be explained during technical discussion or evaluation.

Task 10 – README and Evaluator Setup

- Prepared a `README.md` structure for the Task 29 project.
- Added instructions for running the FastAPI application.
- Used the correct Uvicorn command according to the project structure:

```
uvicorn app.main:app --reload
```

- Documented the main application URLs:
 - API: `http://127.0.0.1:8000`
 - Swagger UI: `http://127.0.0.1:8000/docs`
 - Health check: `http://127.0.0.1:8000/health`
- Added project structure documentation.
- Added dependency installation instructions.
- Added `.env` configuration guidance.
- Added API endpoint descriptions.
- Added example request and response structures.
- Added local Ollama model requirements.
- Added pytest commands for running the tests.
- Added information about the PEFT/LoRA documentation.
- Included guidance to keep hosted API keys private.

Key Learning

Today's main learning was that **building an LLM application is not only about calling a language model**. A reliable LLM service also requires clear separation between the API layer, model-calling logic, provider configuration, model runtime, error handling, testing, and documentation.

I learned how a **shared provider-independent function** such as `ask_model()` can keep the local and hosted endpoints consistent while allowing the underlying provider to change through configuration. This demonstrates the value of abstraction and reduces duplicated inference logic.

I also strengthened my understanding of **OpenAI-compatible APIs**. Different model runtimes and providers can expose compatible interfaces, allowing the same Python client and request structure to communicate with different backends. This makes it easier to switch between local and hosted inference without rewriting the application.

Working with **Ollama** also helped clarify the difference between a model and the software used to serve it. Ollama provides the local runtime and API interface, while Llama 3.2 3B is the actual language model being requested by the application.

The quantization review helped me understand how local LLMs can use lower-precision model representations to reduce resource requirements. I also learned that the application does not necessarily perform quantization itself; the selected model/runtime can provide an already quantized model variant such as **Q4_K_M**.

Finally, preparing the **README, tests, and PEFT/LoRA documentation** reinforced the importance of making an ML/LLM project reproducible and understandable. A technically working service should also provide clear setup instructions, correct execution commands, configuration guidance, testing instructions, and documentation for the concepts used in the implementation.

Blocker:

None